



LOST BOYS CYBER

THREAT INTELLIGENCE • MALWARE ANALYSIS • DETECTION ENGINEERING



CVE-2026-42504 Quadratic complexity in WordDecoder.DecodeHeader in mime

CVE / Vulnerability Threat Intelligence Report

Classification	TLP:CLEAR
Published	2026-06-07
Version	1.1
Author	Lost Boys Cyber
Report Type	CVE / Vulnerability Threat Intelligence Report

TLP:CLEAR | Lost Boys Cyber | CVE-2026-42504 Quadratic complexity in WordDecoder.DecodeHeader in mime - CVE / Vulnerability Threat Intelligence Report

Published: 2026-06-07 | TLP:CLEAR | Version: 1.1 | Author: Lost Boys Cyber

AI Generation: This report was generated with AI assistance and is provided for informational purposes only. All findings, IOCs, detection rules, hunting queries, attribution assessments, and recommendations must be independently reviewed and validated by a qualified security professional before operational use.

Table of Contents

1. Executive Summary
2. Vulnerability Metadata
3. Source Review & Confidence
4. Vulnerability Details
5. Attack Scenarios
6. Threat Landscape & Exploitation Status
7. Affected Products and Exposure
8. Mitigation and Workarounds
9. Detection Engineering
10. Threat Hunting
11. MITRE ATT&CK Mapping
12. Validation / Lab Testing
13. Recommended Actions Summary
14. References

CVE-2026-42504 Quadratic complexity in WordDecoder.DecodeHeader in mime

1. Executive Summary

CVE-2026-42504 is a high-severity denial-of-service issue in Go's ``mime.WordDecoder.DecodeHeader`` logic. Public issue tracking from the Go project and the NVD description indicate that a maliciously crafted MIME header containing many invalid encoded-words can trigger excessive CPU consumption due to quadratic parsing behavior. The bug is relevant anywhere untrusted email or MIME-like headers are parsed with vulnerable Go standard-library code.

The primary operational risk is availability degradation in services that parse attacker-controlled headers at scale, especially mail gateways, mail-processing backends, secure messaging systems, API bridges, and custom Go applications that decode RFC 2047-style encoded words. Although this is not a code execution issue, it can still enable effective service exhaustion when hostile inputs are repeatedly supplied to exposed parsing paths.

2. Vulnerability Metadata

Field	Value
CVE	CVE-2026-42504
Ecosystem	Go standard library
Component	<code>`mime.WordDecoder.DecodeHeader`</code>
Weakness	CWE-407 Inefficient Algorithmic Complexity
CVSS v3.1	7.5 High
Attack Vector	Network
Privileges Required	None
User Interaction	None
Primary Impact	Excessive CPU consumption / denial of service
Public Fix Signal	Go project references change CL 774481 and security issue 79217
Likely Exposure Surface	Mail, messaging, and content-processing services parsing untrusted MIME headers

3. Source Review & Confidence

Source	Contribution	Confidence
Go issue 79217	Primary technical explanation of quadratic behavior and malformed header pattern	High
NVD	Normalized severity, summary, and cross-reference set	High
Go security references / golang-announce	Supports release timing and remediation context	Medium

Confidence is high for the core vulnerability mechanics because the Go security issue clearly explains the parser behavior: repeated invalid encoded-word prefixes cause nearly quadratic work when ``DecodeHeader`` partially consumes malformed

input and resumes parsing. Confidence is moderate on ecosystem exploitation prevalence because the public record focuses on vulnerability mechanics rather than real-world attacks.

4. Vulnerability Details

According to the Go issue, when ``mime.WordDecoder.DecodeHeader`` fails to decode an encoded word, it preserves the undecoded input in the output. The vulnerable behavior arises because the function consumes the initial ``=?`` prefix, appends it, and resumes parsing. A malicious input containing many invalid encoded-word prefixes followed by a terminal ``?=`` can therefore force repeated rescanning and near-quadratic processing.

The NVD describes the practical outcome as excessive CPU consumption when decoding maliciously crafted MIME headers. This makes the issue particularly relevant to services that accept attacker-controlled message headers across high-volume or internet-facing workflows.

5. Attack Scenarios

1. Email gateway or parser exhaustion
 - An attacker sends a stream of messages with malformed encoded-word headers to a service written in Go.
 - Header parsing consumes disproportionate CPU, degrading throughput or causing worker starvation.
2. Abuse of API or message translation services
 - Custom services that transform or inspect MIME-like headers on uploads, inbound webhooks, or relay traffic can be forced into expensive parsing loops.
 - The attacker may not need authentication if parsing occurs pre-auth or during public intake flows.
3. Multi-tenant service disruption
 - In SaaS environments where a shared parser handles multiple tenants, a single hostile input stream can create noisy-neighbor denial-of-service effects.

6. Threat Landscape & Exploitation Status

Public reporting currently centers on the Go project disclosure, fix workflow, and NVD publication. We did not identify a KEV listing or public evidence of widespread exploitation in the source set reviewed here. Even so, algorithmic-complexity bugs in parser paths deserve prompt remediation because they are low-friction for adversaries to weaponize once a public proof pattern exists.

Organizations should treat externally reachable Go-based message processing systems as the highest priority patch targets, especially when they process large volumes of untrusted email or MIME header content.

7. Affected Products and Exposure

Product / Surface	Exposure Consideration	Priority
Go applications using vulnerable stdlib releases	Directly affected if they call <code>`mime.WordDecoder.DecodeHeader`</code> on untrusted data	High
Email ingestion pipelines	High likelihood of hostile MIME header exposure	Immediate
API bridges, notification services, message relays	May parse attacker-controlled encoded words outside traditional mail contexts	High
Internal-only batch processors	Lower exposure but still patch if they process partner or inbound third-party content	Medium

Defenders should inventory applications built on affected Go releases, identify whether they parse encoded words or MIME headers from untrusted sources, and prioritize internet-facing or business-critical parsing services.

8. Mitigation and Workarounds

4. Upgrade to a Go release containing the fix referenced by the Go security advisory / issue 79217 workflow.
5. Where immediate upgrades are not possible, reduce exposure by rejecting or rate-limiting suspiciously malformed headers at upstream gateways.
6. Apply CPU and worker-level safeguards for parsing-heavy services to reduce blast radius from hostile inputs.
7. Add observability for parser error spikes, request latency growth, and abnormal CPU utilization in mail-processing components.
8. Review application code to determine where ``mime.WordDecoder.DecodeHeader`` or adjacent mail-decoding helpers are called with attacker-controlled input.

9.1 Detection Opportunities

Signal	Rationale	Data Source
Sudden CPU spikes in Go-based mail or message parsers	Core observable outcome of algorithmic complexity abuse	Host telemetry, APM
Repeated parser errors or malformed MIME decoding warnings	May surface hostile header attempts	Application logs
Increased processing latency for inbound email / message jobs	Useful early symptom before outright outage	Queue metrics, job telemetry
High-volume repeated inbound messages from a small sender set	Common delivery pattern for DoS attempts	Mail gateway, SMTP logs

9.2 Detection Query

```
// Surface CPU stress and suspicious processing patterns on likely Go-based parsing services
DeviceProcessEvents
| where Timestamp > ago(14d)
| where FileName has_any ("go", "mail", "smtp", "worker") or ProcessVersionInfoProductName has "Go"
| summarize Launches=count(), Accounts=make_set(InitiatingProcessAccountName, 10) by DeviceName,
FileName, FolderPath
| join kind=leftouter (
    DeviceNetworkEvents
    | where Timestamp > ago(14d)
    | where LocalPort in (25, 465, 587, 993, 995, 443)
    | summarize Connections=count(), DistinctIPs=dcount(RemoteIP) by DeviceName,
InitiatingProcessFileName
) on $left.DeviceName == $right.DeviceName and $left.FileName == $right.InitiatingProcessFileName
| order by Connections desc nulls last
```

9.3 Exposure / Triage Query

```
// Identify systems likely processing mail or MIME-like content with Go binaries
DeviceFileEvents
| where Timestamp > ago(30d)
| where FileName endswith ".go" or FolderPath has_any ("/go/", "golang", "mail", "smtp")
| summarize ExamplePaths=make_set(FolderPath, 10), FileTouches=count() by DeviceName
| order by FileTouches desc
```

10.1 Hunt Leads

- Review mail gateways and custom Go services for periods of elevated CPU with no corresponding traffic growth from legitimate senders.
- Pivot from malformed MIME header logging into sender IPs, envelope addresses, and tenant identifiers.
- Identify external services that ingest partner email or notifications and may parse encoded words before authentication or anti-abuse checks.

10.2 Hunt Query

```
// Find clusters of inbound connectivity to message-processing systems that may align with parser abuse
DeviceNetworkEvents
| where Timestamp > ago(14d)
| where LocalPort in (25, 465, 587, 993, 995)
| summarize EventCount=count(), DistinctDestinations=dcount(DeviceName), SampleUrls=make_set(RemoteUrl, 10)
    by RemoteIP, bin(Timestamp, 1h)
| where EventCount > 100
| order by EventCount desc
```

11. MITRE ATT&CK Mapping

Tactic	Technique	Relevance
Impact	T1499 Endpoint Denial of Service	Excessive CPU use can degrade or halt service availability
Initial Access / Delivery Context	T1566 Phishing or message delivery paths	Relevant when hostile headers arrive through email or message ingestion workflows

12. Validation / Lab Testing

9. Confirm vulnerable Go runtimes or affected binaries are upgraded.
10. In a non-production lab, test parser behavior with malformed encoded-word samples to validate performance improvements after upgrade.
11. Validate alerting for sustained CPU spikes, queue growth, and parser exceptions in mail-handling components.
12. Confirm upstream mail gateways or proxies can reject obviously malformed header floods before they reach parsing workers.

13. Recommended Actions Summary

Priority	Action
Immediate	Upgrade Go runtimes and affected applications to fixed releases
High	Identify all services parsing untrusted MIME headers or encoded words
High	Add or tune alerts for CPU spikes, queue latency, and parser errors
Medium	Rate-limit or reject malformed headers at upstream gateways where feasible
Medium	Stress-test critical parsing services after patching to confirm improved resilience

14. References

13. Go security issue 79217 for `mime.WordDecoder.DecodeHeader`
14. NVD entry for CVE-2026-42504
15. Go security / release references including change CL 774481 and GO-2026-5038